

多核堆栈处理器中多核调度机制研究与设计

刘自昂¹ 周永录^{1,2*} 代红兵^{1,2} 刘宏杰^{1,2}

¹(云南大学信息学院 云南 昆明 650500)

²(云南省高校数字媒体技术重点实验室 云南 昆明 650223)

摘要 多核堆栈处理器作为 Forth 领域的研究热点之一,目前已取得了一定的研究进展,但多核堆栈处理器面临着缺乏高效 Forth 系统支撑的问题。针对 Forth 多核堆栈处理器的特性,研究并设计一种多核调度机制,该多核调度机制的多核调度算法使用全局调度,多核任务调度算法使用可变时间片轮转调度算法和 EDF(Earliest Deadline First)调度算法,重点解决多核堆栈处理器平台的 Forth 任务调度问题。实验表明,多核调度机制能够在基于 FPGA 实现和工作于 100 MHz 频率的多核堆栈处理器上可靠运行,实现任务的正确调度,普通任务响应时间最低为 0.5 ms,实时任务的平均响应最长为 9.36 μ s。

关键词 多核堆栈处理器 Forth 系统 多核调度机制 全局调度 可变时间片轮转调度算法 EDF 调度算法

中图分类号 TP316

文献标志码 A

DOI:10.3969/j.issn.1000-386x.2025.09.035

RESEARCH AND DESIGN OF MULTICORE SCHEDULING MECHANISM IN MULTICORE STACK PROCESSOR

Liu Ziang¹ Zhou Yonglu^{1,2*} Dai Hongbing^{1,2} Liu Hongjie^{1,2}

¹(School of Information, Yunnan University, Kunming 650500, Yunnan, China)

²(Digital Media Technology Key Laboratory of Universities in Yunnan, Kunming 650223, Yunnan, China)

Abstract As one of the research hotspots in the field of Forth, multicore stack processors have made some research progress, but multicore stack processors face the problem of lacking efficient Forth system support. A multicore scheduling mechanism is studied and designed for the characteristics of Forth multicore stack processors. The multicore scheduling algorithm of this multicore scheduling mechanism used global scheduling, and the Forth task scheduling algorithm used variable time slice rotation scheduling algorithm and earliest deadline first (EDF) scheduling algorithm, focusing on solving the Forth task scheduling problem of multicore stack processor platforms. Experiments show that the multicore scheduling mechanism can run reliably on a multicore stack processor based on FPGA implementation and operating at 100MHz frequency to achieve correct scheduling of tasks, with a minimum response time of 0.5 ms for common tasks and a maximum average response of 9.36 μ s for real-time tasks.

Keywords Multicore stack processor Forth System Multicore scheduling mechanism Global scheduler Variable time slice rotation scheduler algorithm Earliest deadline first

0 引言

1971 年用于控制 11 米射电望远镜并取得了巨大成功,因此天文学界开始广泛应用 Forth 语言,国际天文学联合会于 1976 年将 Forth 语言作为其标准语言^[1]。

Forth 语言由美国计算机科学家 Moore 发明,在

Forth 语言是一种完全模块化的高级程序设计语言,同

时又是一种操作系统,它具有程序小、执行快、交互性强、易于移植的特点^[2-3]。Forth 中所有操作都基于数据堆栈和返回堆栈,例如加法操作,Forth 直接使参数堆栈的栈顶和次栈顶的数值相加,结果再放回栈顶,这个操作只需要一个指令周期,这使得连续的逻辑运算操作和跳转操作变得十分迅速。Forth 语言在寄存器型处理器上运行时,参数堆栈和返回堆栈的数据均存放在内存之中,Forth 语言每次访问堆栈上的数据时都要多次访问内存和处理器内的寄存器,浪费了宝贵的系统资源,为解决这个问题,堆栈处理器应运而生。堆栈处理器由于任务切换快、切换开销低、资源占用少等优良特性,在嵌入式领域有很好的表现,出现了多款经典的堆栈处理器,如 MSL16^[4]、EP32^[5]、b16^[6]等。堆栈处理器极大地提高了 Forth 语言的执行效率,特别是在实时处理上,在相同工作频率的环境下,基于单核堆栈处理器的实时 Forth 系统比基于寄存器处理器的实时 Forth 系统任务响应时间快了 46 倍,基于单核堆栈处理器的实时 Forth 系统在 100 MHz 的工作频率下任务响应时间可达到 $7.47 \mu\text{s}$ ^[7-8]。由此可见 Forth 系统和堆栈处理器在嵌入式领域具有巨大的应用潜力。

随着嵌入式环境日益复杂,单核堆栈处理器的性能已经不能很好地满足应用需求,多核堆栈处理器已成为 Forth 领域的研究热点之一。GreenArrays 公司推出的 GA144 片上多处理机芯片,该芯片拥有 144 个内核,每个内核都是一个独立的 F18A 堆栈计算机,GA144 的性能达到了 960 亿条指令/秒,最大功耗仅为 650 mW^[9-11]。南京航空航天大学的何凯成^[12]实现了一个双核堆栈处理器,它的从核仅与主核通信,辅助主核进行任务处理,主核则负责与处理器的其他模块进行通信,主核和从核都使用了多级流水线设计,处理器工作频率达到了 209 MHz。

多核处理器分为同构多核处理器和异构多核处理器。同构多核处理器的每个内核共同运行一个操作系统,共享系统中所有资源,每个内核上的任务统一由操作系统进行分配,这种运行方式被称为对称多处理 SMP(Symmetrical Multi-Processing)^[13]。本文多核调度机制主要针对自研的多核堆栈处理器设计,该多核堆栈处理器为同构内核,因此对 SMP 多核调度进行研究。

在多核环境下,Linux 系统的多核调度算法采用划分调度方式,每个内核都有属于自己的调度队列,通过周期性调用负载均衡任务来保证各个内核的均衡使用,桌面 Linux 系统中,进程调度器使用时间片轮询 RR(Round-Robin)和先进先出 FIFO(First In First Out)算法对实时进程调度,使用完全公平调度 CFS(Com-

pletely Fair Scheduler)算法对普通进程进行调度;嵌入式 Linux 系统中,使用最早截止时间优先 EDF 算法、单调速率 RM(Rate Monotonic)算法、最低松弛度优先 LLT(Least Laxity First)等算法或这些算法的改进版本对任务进行调度^[14]。郑星^[15]对 FreeRTOS 在 ARM 平台的 SMP 系统方案进行了研究,该系统的多核调度采用全局多级就绪队列的方式设计任务调度队列,任务调度算法使用优先级抢占和时间片轮转的方式对任务进行调度。任晓宇等^[16]设计了一种基于核集的多核嵌入式调度方法,该方法中多核调度策略既未采用全局调度方法,也未采用划分调度方法,而是综合这两种调度的特点,采取半划分调度方法,使得操作系统中既有全局调度队列又有划分调度队列,但半划分调度方式仍存在负载不均衡的缺陷,所以任晓宇等在此基础上对半划分调度进行改进,根据任务特点的不同,为任务划分多个内核,任务只能在被划分的内核上运行。

GA144 芯片与传统多核芯片不同,它以大规模并行和流水线为目标而设计,该芯片上的 Forth 任务在运行前会进行分解,通过 arrayForth 系统把分解好的多个子任务写入一个个 F18A 内核中,使每个 F18A 内核都可以看作一个“Forth 词”或者说是一个“函数”。在大规模并行模式下,每个 F18A 内核都作为一个相对独立的子任务,并行执行;在流水线模式下,任务在 GA144 上运行过程就是按顺序在一个个具有不同“Forth 词”的 F18A 堆栈计算机上“游走”,这种方式就像数据流图或流水线图映射到了计算机阵列上,无论是规模并行模式还是流水线模式,一个或多个 Forth 任务同时在 GA144 的内核阵列上各自占据一块阵列空间,任务运行时,不会发生任务切换,所以 GA144 的多核调度机制不存在。除了 GA144,现有的多核 Forth 处理器大多针对特定的应用,每个内核都运行着一个 Forth 系统,系统与系统之间相互独立,或者多个内核共同运行一个 Forth 系统,但可运行的 Forth 任务数与内核数相同,因而这两种方式都不存在传统意义上的多核调度机制。

现有的多核堆栈处理器研究与产品设计主要有 GA144 和文献[12]两种,其 Forth 多核调度机制均未公开。作者所在团队在多核堆栈处理器架构上进行了探索,已取得一定的阶段性进展,但目前仍缺乏可支撑的高效 Forth 系统,所以本文拟结合多核堆栈处理器的特点,借鉴传统多核调度机制,对 Forth 多核堆栈处理器的多核调度机制进行研究与设计,以为多核堆栈处理器提供高效的 Forth 系统支撑。

1 自研 Forth 多核堆栈处理器

本文 Forth 多核调度机制基于自研的 Forth 多核堆栈处理器,该处理器的基本结构如图 1 所示,包含内核组、中断控制器、内核控制器、临界资源仲裁器、数据总线、指令总线、IO 总线、多端口存储器、外设组等部件。

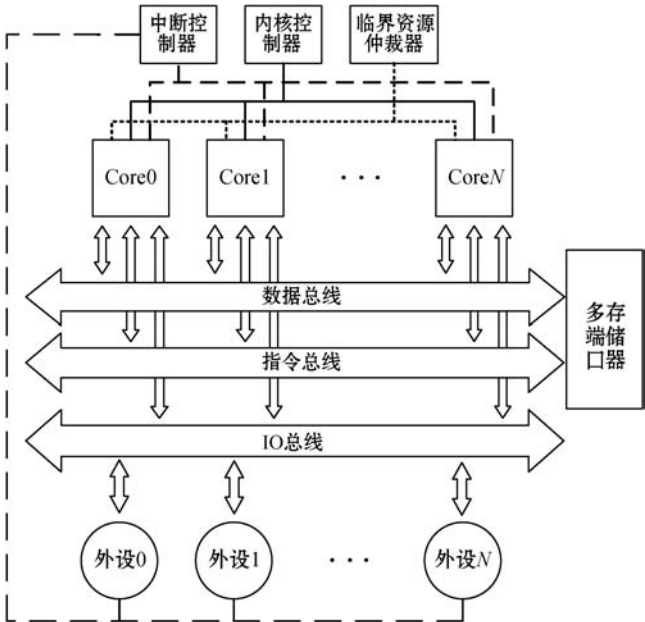


图 1 自研多核堆栈处理器总体结构

每个处理器内核中包含了一个硬件参数堆栈和一个硬件返回堆栈,两个堆栈的深度均是 64,双堆栈、堆栈指针寄存器、程序计数器 PC 等模块存储的数据需要在任务切换时被保存。每个处理器内核都拥有一个自己的专用计数器,并有相关的指令控制计数器,内核可以使用该专用计数器进行计时或计数操作,例如为内核上任务的时间片计时,当任务时间片耗完后,这个计数器可以为所在内核触发一个不受中断控制器控制硬件中断,该中断的中断服务程序地址由用户设置。

中断控制器主要用于响应外设组的中断,并将中断分配给合适的内核去响应。中断控制器内置一块用于存储中断向量的存储区,根据不同的中断源将中断服务程序的地址送入合适的内核去响应中断,加快中断响应的速度。中断控制器还响应全局时间计时器的定时中断,全局时间计时器与内核的专用计数器不同,它用于计时处理器的运行时间,并允许周期性触发定时中断,完成某些必需的操作。

内核控制器主要用于对内核开启和关闭进行控制,Forth 系统可以通过相关指令来实现对多个内核的控制,当任务较多时可以开启多个内核去执行任务,任务较少时可以关闭部分内核以降低功耗。

临界资源仲裁器是一种对 Forth 系统中临界资源

加锁的部件,与软件结合可以实现自旋锁,用于保证系统中独占资源的正确访问,如任务的调度队列等。

目前,已在 FPGA 上实现四核、八核的堆栈处理器,内部不存在流水线,指令周期为单周期,工作频率为 100 MHz。内核间以共享总线方式进行通信,所有内核共享一个内存,其多任务性能分别约为单核堆栈处理器的 3.5 倍和 7 倍,功耗分别约为单核堆栈处理器的 1.3 倍和 2.2 倍。

2 Forth 多核调度机制

2.1 多核调度策略

多核调度主要有三种调度方式,分别是全局调度、划分调度和半划分调度。全局调度方式是使所有的处理器内核共享一个或多个就绪队列,内核需要切换任务时只需要从就绪队列中提取一个任务来执行,因此每个内核的负荷总是均衡的,可充分地利用内核资源,系统中运行的任务不知道自己下一次会在哪一个内核上运行,在最坏的情况下任务每次分配的内核都与上一次不一样,任务的反复迁移会造成缓存命中率下降,降低了系统性能。划分调度方式是使每个内核都有自己的调度队列,内核每次切换任务只需要从自己的调度队列选出一个任务执行,当自己的调度队列中没有任务时则从其他内核的调度队列拉取一部分合适的任务去执行,这种策略不会使任务在内核间频繁迁移,缓存命中率得到了保证,但是会发生每个内核的调度队列任务数差距很大的问题,降低了处理器内核的总体利用率,需要每隔一段时间去进行一次负载均衡。半划分调度方式是全局调度和划分调度的折中策略,这种策略使系统中有全局队列和划分队列,其中划分队列有一定的任务数限制,确保全局队列中有任务被用来调节负载,虽然在一定程度上缓解了缓存命中率低和负载不均衡的问题,但是在嵌入式领域许多任务因其特性需要与内核进行绑定,仍会出现负载不均衡的问题。

Forth 语言是一种局部性原理较弱的程序设计语言,它独特的穿线码技术使得 Forth 任务在执行的过程中需要不断跳转,跳转的过程就是解决问题的过程,因此 Forth 任务在传统的 Cache 上执行性能较低,所以在 Forth 堆栈处理器中一般采用 Forth 缓存。一个功能相对完整的多任务 Forth 系统大小约为几十 KB,完全可以将整个 Forth 系统放入处理器的高速缓存中,在运行的过程中不需要数据的置换,大大降低了缓存的复杂度,我们将这种高速缓存称之为 Forth 缓存。因为

Forth 任务在 Forth 多核堆栈处理器的内核间进行迁移不会出现命中率低的问题,所以划分和半划分调度策略在多核多任务 Forth 系统中不如全局调度策略效率高。本文的多核调度策略采用全局调度,多个内核共同竞争全局就绪队列。

2.2 Forth 任务调度策略

本文的多核调度机制提供两种 Forth 任务调度方式:带优先级的可变时间片轮转调度和 EDF 调度,分别针对普通 Forth 任务和实时 Forth 任务的调度。这两种调度方式均以时间片划分为基础对任务进行调度,每个任务每次可以运行一个时间片,利用处理器内核的计数器对运行在内核上的任务时间片进行计时,任务的时间片耗完后由该内核的计数器发出中断,内核根据系统内其他任务的信息去切换一个任务或继续运行任务。Forth 任务调度策略定义了一个系统变量 `time_task_cnt`,用来记录 Forth 系统中实时的 Forth 任务个数,当系统发生任务切换中断时,根据 `time_task_cnt` 变量的值来选择带优先级的可变时间片轮转调度算法和 EDF 调度算法去调度任务,当 `time_task_cnt > 0` 时选择 EDF 调度算法去调度实时任务,当 `time_task_cnt ≤ 0` 时选择带优先级的可变时间片轮转调度算法去调度普通任务,如图 2 所示。实时 Forth 任务优先程度比普通 Forth 任务高,在任务调度时优先调度实时 Forth 任务。

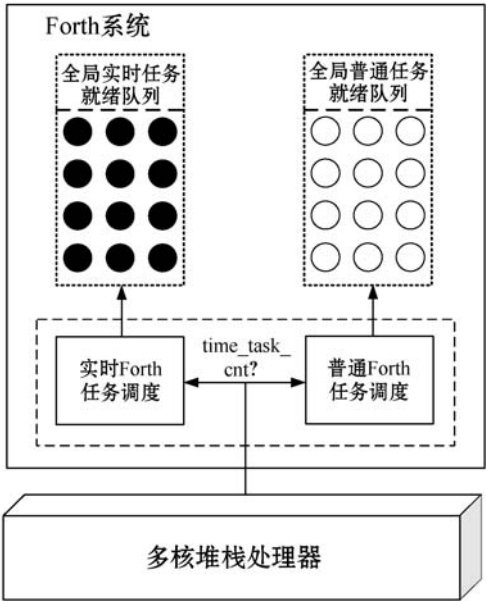


图 2 Forth 任务调度策略

2.2.1 普通 Forth 任务调度策略

嵌入式系统中的每个任务通常都有自己的优先级,在优先级抢占调度算法中总是选择优先级最高的任务投入运行,低优先级任务需要长时间等待高优先级任务执行完毕才能得到执行,这使得低优先级任务

非常容易出现“饥饿”现象。若使用时间片轮转调度算法让每个任务运行一个同等大小的时间片,则可能出现一些任务需要获得更长的 CPU 时间去执行需求的问题。

为解决上述问题,采用一种带优先级的可变时间片轮转调度算法对普通任务进行调度,该算法以时间片轮转调度算法为基础,每个任务轮转执行一个时间片,使用任务优先级占所有任务优先级之和的比值来确定任务在总调度周期内执行的时间片大小,这种任务调度方法只需要使用一个全局普通任务就绪队列来组织就绪态的普通任务。总调度周期指的是所有就绪的普通任务运行一次的时间。时间片大小的计算方法为:总调度周期 × 当前任务优先级 / 所有任务优先级之和。

这种调度方法还存在一个问题,就是随着进入系统的普通任务增多,在总调度周期不变的情况下,每个任务分到的时间片会趋于无限小,引发频繁的任务切换。本文的解决方法是,为系统中的任务确定一个任务最小运行时间,所有的任务在一次调度中运行的时间不允许比任务最小运行时间短,因为任务的一次运行时间是根据优先级占比计算得出,所以总调度周期也必须做出相应的调整,保证每个任务的一次执行时间比任务最小运行时间长。总调度周期的确定方法是将所有任务优先级之和 / 最小任务优先级 × 任务最小运行时间,当系统中只有一个普通任务在运行时使用上述式子计算得到的总调度周期与任务最小运行时间相同,会导致频繁发生任务切换中断,所以规定总调度周期最小为 2 ms。在任务启动、结束、阻塞和优先级调整时对总调度周期进行调整,即所有任务优先级之和改变时对总调度周期进行调整。普通任务的优先级为 0 ~ 12 级,由于任务优先级的值存在 0,所以在计算时所有优先级的值加 1。

普通 Forth 任务调度过程如图 3 所示,首先要判断变量 `time_task_cnt` 是否大于 0,如果大于则转去执行实时 Forth 任务调度,反之则判断任务是否因任务运行结束而触发任务切换中断,若任务运行结束则回收任务资源,并判断系统中是否还有就绪的普通任务,若无则关闭内核,反之则取出任务执行,若任务未运行结束,则判断系统中是否还有就绪态的普通任务,若无则继续执行当前任务,反之则保存当前任务的执行现场,将任务插入到全局普通任务就绪队列的队尾,再从该队列的队头取出一个新的就绪任务,恢复现场,计算任务的时间片大小,然后继续执行该任务。

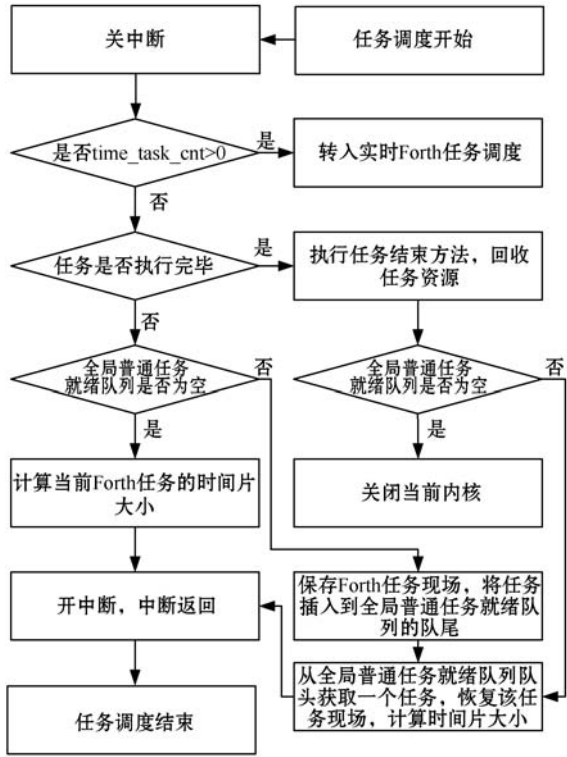


图 3 普通 Forth 任务调度过程

2.2.2 实时 Forth 任务调度策略

在堆栈处理器中,使用参数堆栈和返回堆栈代替传统处理器内的通用寄存器和部分专用寄存器,因为堆栈先进后出的特性,使得堆栈内上层数据与下层数据互不干扰,所以在堆栈上可以发生任务的堆叠,在堆叠前只需保存当前任务的专用寄存器,例如堆栈指针寄存器、程序计数器 PC 等,新任务直接在原堆栈上运行不会出现异常,这种任务堆叠方式非常适合用于急迫任务对非急迫任务的抢占。随着任务堆叠增多,堆栈存在溢出的风险,为了避免堆栈溢出,需要对任务的堆叠数量做出一定的限制。当堆栈深度达到 32 时,几乎所有任务都不会发生堆栈溢出的情况^[17],我们自研多核堆栈处理器内核的堆栈深度为 64,所以限制任务堆叠数为 2。

实时 Forth 任务的调度采用 EDF 调度算法,EDF 调度算法每次调度都选择截止时间最早的任务,本文使用一个全局实时任务就绪队列组织就绪的实时任务,队列中的任务按照最早截止时间排序,截止时间早的任务排在队头,截止时间晚的任务排在队尾,实时任务没有优先级,该类任务优先于普通任务调度。在任务堆叠数小于 2 时,采用时间片轮转的方式执行实时任务,每当时间片耗完后根据系统内是否有更早截止时间的实时任务来决定当前任务是否继续执行;在任务堆叠数等于 2 时,实时任务将不采用时间片的方式执行,而是一次性将任务执行完毕。一个实时任务被更急迫的实时任务压到堆栈下层时,该任务不会回到

全局实时任务就绪队列,也不会被执行,而是等待上层任务运行完毕后再继续执行。因为实时任务从全局实时任务就绪队列出队后不会再回到该队列,所以全局实时任务就绪队列的排序不会发生在实时任务切换的过程中。

实时 Forth 任务的调度过程如图 4 所示,首先判断系统变量 time_task_cnt 是否大于 0,如果不大于则转去执行普通 Forth 任务调度,若大于则判断当前任务是否是实时任务,若是实时任务则判断全局实时任务就绪队列中是否有比当前实时任务更急迫的实时任务,若有,则取出该任务,设置该任务 PC 并保存被抢占任务的 PC 和堆栈指针等寄存器,再根据当前内核的任务堆叠数来控制是否对该任务时间片计时;反之,则继续执行当前任务。若当前任务是普通任务,则需要保存现场,再获取实时任务。普通任务切换到实时任务时保存现场是为了能够多响应一次急迫的实时任务。由于实时任务的结束相关步骤与普通任务的结束相关步骤相似,故省略,但在全局实时任务就绪队列为空时不关闭内核,而是转入普通 Forth 任务调度步骤。

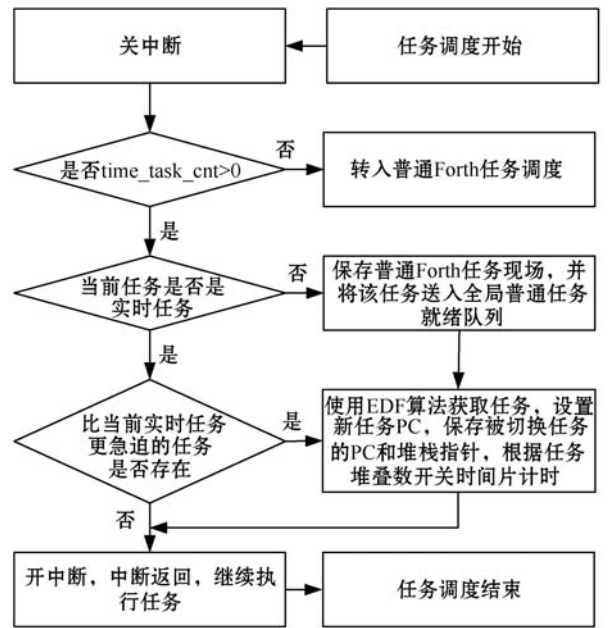


图 4 实时 Forth 任务调度过程

3 实验验证

基于本文多核调度机制的多任务 Forth 系统采用 Forth 语言编写实现,通过 GForth 系统进行交叉编译生成,连同自研的多核堆栈处理器 HDL 设计所生成的位流文件,一并烧写到 Xilinx XC7A100tfgg676 FPGA 芯片中。实验证明,多核处理器及多任务 Forth 系统可在 100 MHz 的工作频率下稳定运行。

为验证本文设计的多核调度机制的性能,采用运

行多个斐波那契数列求解算法任务方法来进行实验验证,分析任务运行的时间开销。斐波那契数求解任务的内容是使用递归的方法求取第 20 个斐波那契数。为使多核堆栈处理器发挥最大性能,斐波那契数求解任务运行的数量应为多核堆栈处理器内核数的整数倍,所以将任务的运行个数确定为 24,分别以普通任务和实时任务两种方式运行,普通任务的优先级使用 6 级,在没有其他任务运行的情况下,分别在单核、四核和八核堆栈处理器的实现实例上执行 24 个任务,获取其时间开销,如表 1 所示。

表 1 24 个斐波那契数求解任务执行时间 单位:ms

任务	单核 堆栈处理器	四核 堆栈处理器	八核 堆栈处理器
普通任务	86.57	23.97	13.57
实时任务	70.65	20.01	11.15

由表 1 可知,单核、四核和八核堆栈处理器运行 24 个斐波那契数求解任务所花费时间的比值与它们的内核数的比值有直接关系,八核堆栈处理器运行 24 个任务的速度约为单核堆栈处理器的 7 倍,约为四核堆栈处理器的 2 倍;四核堆栈处理器执行 24 个任务的速度约为单核堆栈处理器的 3.5 倍。24 个斐波那契数列求解任务在多核堆栈处理器上成功运行,并且这些任务的总运行时间符合自研多核堆栈处理器的多任务执行性能的比值,说明本文设计的多核调度机制可以有效调度多核去执行任务。在任务执行效率上,实时任务调度比普通任务调度提高了 17.8% 左右,这是因为实时任务切换时不会保存堆栈,并且是一次执行完毕,节省了任务切换的开销。

表 2 中分别展示了单个斐波那契数列求解任务分别作为普通任务和实时任务在有 16 个其他 6 级优先级的普通任务运行情况下的执行时间,反映了普通任务调度和实时任务调度对任务执行效率的影响。在普通任务调度中,任务优先级值越大,优先级越高,获得的运行时间越多,任务执行耗费的时间越短,同时优先级为 0 的任务得到了运行,说明了普通任务调度策略解决了低优先级任务会“饿死”的问题,并且保证了高优先级任务可以得到更多的运行时间。在实时任务调度中,实时任务的出现直接抢占了一个内核,以最快的速度执行任务,表 2 中可见斐波那契数列求解任务在作为实时任务时,任务执行时间均在 3.40 ms 左右,说明了实时 Forth 任务调度可以在混合任务的环境下对实时任务进行正确调度。

表 2 不同优先级下任务执行时间 单位:ms

任务	任务 优先级	单核 堆栈处理器	四核 堆栈处理器	八核 堆栈处理器
普通任务	0	214.17	48.83	22.79
	6	57.96	16.26	8.72
	12	21.41	6.75	4.59
实时任务	—	3.41	3.40	3.40

在 Forth 任务切换时,不同的任务使用的堆栈大小不同,它们的切换时间也不同,通过 Vivado 软件对多任务 Forth 系统和多核堆栈处理器进行仿真,可得本文多核调度机制任务的切换时间。普通任务间的任务切换时间平均为 97.97 μ s;普通任务切换到实时任务的时间平均为 5.13 μ s。实时任务的切换有两种方式:(1) 实时任务被更紧迫的实时任务抢占,这种方式的 任务切换时间平均为 2.81 μ s;(2) 实时任务执行完毕,切换其他实时任务的时间平均为 7.43 μ s,这个过程中需要对任务资源进行回收,所以要比直接的任务切换花费更多的时间。

普通任务的调度算法本质上是时间片轮转,所以普通任务的响应时间与运行的任务个数和内核数有关,普通任务的响应时间不易确定,单核堆栈处理器普通任务的响应时间最短不小于 2 ms,四核、八核堆栈处理器在每个内核都有任务运行的情况下,普通任务的响应时间最短不小于最小任务执行时间,为 0.5 ms。实时任务的调度算法是 EDF 算法,实时任务只能被更紧迫的实时任务抢占,实时任务的启动一般是由中断服务程序完成,中断返回时会调用一次任务切换中断,所以任务响应时间则是任务启动时间与任务切换时间之和,最长为 9.36 μ s。

4 结 语

本文针对自研多核堆栈处理器的特性,研究并设计多核堆栈处理器上的多核调度机制,包括多核调度策略和 Forth 任务调度策略,Forth 任务调度策略根据不同的任务分为实时任务调度和普通任务调度。由本文多核调度机制所实现的多任务 Forth 系统,在单核、四核和八核堆栈处理器上成功运行,在 Forth 任务的切换和响应性能方面具有良好的表现,为面向多核堆栈处理器的多任务 Forth 系统的设计与应用提供一定的理论支撑。本文设计的多核调度机制不以实时为目标,仅对多核 Forth 系统的实时性进行一定的探索,其实时调度算法仍存在一些问题,比如 EDF 算法的 Dhall 效应没有给出解决方案。本文多核调度机制与

面向单核堆栈处理器的实时 Forth 系统^[7]相比,实时性有一定差距,这是因为文献[7]所实现的 Forth 系统设计以实时性为目标,针对单核堆栈处理器而设计,不存在任务调度队列的竞争,为运行任务申请内存空间时采用顺序分配算法,本文采用动态分区分配,单核堆栈处理器在访问内存时不存在多核竞争的情况,而多核堆栈处理器存在多核竞争的情况,文献[7]的单核堆栈处理器为了更好地配合实时 Forth 系统,在处理器内设置了多套任务硬件堆栈,自研多核堆栈处理器内核,为了节约功耗和资源,仅设置一套堆栈。在本文多核调度机制研究的基础上,支持强实时性的多核堆栈处理器以及多核多任务实时 Forth 系统将是我们的下一步的研究与设计目标。

参 考 文 献

- [1] Rather E D, Colburn D R, Moore C H. The evolution of forth[C]//History of Programming Languages, 1993:625 – 670.
 - [2] 张怀宁. FORTH 语言教程[M]. 北京:机械工业出版社,1989.
 - [3] 代红兵,周永录,安红萍,等. 基于 Forth 虚拟机的嵌入式多任务操作系统体系架构研究[J]. 计算机应用研究, 2019,36(2):476 – 480.
 - [4] Leong P H, Tsang P K, Lee T K. A FPGA based forth microprocessor[C]//IEEE Symposium on FPGAs for Custom Computing Machines, 1998:254 – 255.
 - [5] Hjrtland E, Chen L. EP32-a 32-bit forth micorprocessor [C]//Conference on Electrical & Computer Engineering, 2007:518 – 521.
 - [6] Paysan B. b16-A forth processor in an FPGA[EB/OL]. [2022 – 05 – 24]. <https://www.doc88.com/p-686307649508.html>.
 - [7] 郭金辉,刘宏杰,代红兵,等. 基于堆栈处理器的周期多任务调度机制研究与实现[J]. 计算机应用研究, 2021,38(9):2752 – 2756.
 - [8] 黄忠建,代红兵,王蕾. 嵌入式 Forth 操作系统周期性研究[D]. 昆明:云南大学,2019.
 - [9] GreenArrays. DB002-G144A12 chip reference [EB/OL]. [2022 – 05 – 24]. <https://www.greenarraychips.com/home/documents/greg/DB002-110705-G144A12.pdf>.
 - [10] GreenArrays. DB001-F18A technology reference[EB/OL]. [2022 – 05 – 24]. <https://www.greenarraychips.com/home/documents/greg/DB001-221113-F18A.pdf>.
 - [11] GreenArrays. PB002-greenarrays architecture[EB/OL]. [2022 – 05 – 24]. <https://www.greenarraychips.com/home/documents/greg/PB002-100822-GA-Arch.pdf>.
 - [12] 何凯成. 基于堆栈式多核处理器的嵌入式系统的研究与实现[D]. 南京:南京航空航天大学,2015:1 – 75.
 - [13] 董延军,项涛. 多核嵌入式操作系统及板级结构探讨[J]. 信息通信, 2018(12):145 – 147.
 - [14] 邓竹莎. 面向多处理器结构的嵌入式 Linux 系统研究与实现[D]. 成都:电子科技大学,2006.
 - [15] 郑星. FreeRTOS 在 ARM 平台的 SMP 系统方案研究与实现[D]. 北京:北京邮电大学,2021.
 - [16] 任晓宇,周霆,王立. 一种基于核集的多核嵌入式周期调度方法[J]. 单片机与嵌入式系统应用, 2022,22(7):3 – 6.
 - [17] Koopman P J. Stack Computers: The new wave[M]. Halsted Press, 1989:139 – 146.
-
- (上接第 195 页)
- [23] Wang X, Chen Y, Zhu W. A survey on curriculum learning [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021,44(9):4555 – 4576.
 - [24] Kingma D, Ba J. Adam: A method for stochastic optimization[EB]. arXiv:1412.6980, 2014.
 - [25] Xia G, Hu J, Hu F, et al. AID: A benchmark data set for performance evaluation of aerial scene classification [J]. IEEE Transactions on Geoscience and Remote Sensing, 2017,55(7):3965 – 3981.
-
- (上接第 247 页)
- [27] Schlemper J, Oktay O, Schaap M, et al. Attention gated networks: Learning to leverage salient regions in medical images[J]. Medical Image Analysis, 2019,53:197 – 207.
 - [28] Hatamizadeh A, Tang Y C, Nath V, et al. UNETR: Transformers for 3D medical image segmentation[C]//IEEE Winter Conference on Applications of Computer Vision, 2022: 574 – 584.
 - [29] Wang W H, Xie E Z, Li X, et al. Pvtv2: Improved baselines with pyramid vision transformer [J]. Computational Visual Media, 2022,8(3):415 – 424.
 - [30] Huang X H, Deng Z F, Li D, et al. Missformer: An effective medical image segmentation transformer[EB]. arXiv: 2109.07162, 2021.
 - [31] Ji Y F, Zhang R M, Wang H J, et al. Multi-compound transformer for accurate biomedical image segmentation[C]//International Conference on Medical Image Computing and Computer-Assisted Intervention, 2021:326 – 336.
 - [32] Bernard O, Lalande A, Zotti C, et al. Deep learning techniques for automatic MRI cardiac multi-structures segmentation and diagnosis: Is the problem solved? [J]. IEEE Transactions on Medical Imaging, 2018, 37(11):2514 – 2525.
 - [33] Fu S H, Lu Y, Wang Y, et al. Domain adaptive relational reasoning for 3d multi-organ segmentation[C]//International Conference on Medical Image Computing and Computer-Assisted Intervention, 2020:656 – 666.